Week 5 — Engineer Features and Reduce Dimensionality

===== WEEK 5 START =====

Project: Airline Passenger Demand Forecasting, BTS T-100 Domestic Segment

Week 5 objectives

- Generate cross-field domain features from current-month carrier-route data.
- Decide whether synthetic augmentation is appropriate.
- Reduce the numeric feature space by principal component analysis (PCA) **fitted on training only**.

> A forecast for month *t+1* uses observed information from month *t*. The exercise assumes the month-*t* BTS aggregates are available at the time the forecast is issued. Operational reporting latency must be checked before deployment.

1. Environment / reusable code

- All project logic is in `Src/week5_src_features_dimensionality.py`.
- Notebook imports the same module so that it and the standalone Python program have identical formulas and preprocessing.

In [1]:
```
from pathlib import Path
import importlib.util
import sys
from IPython.display import display, Image


def find_project_root(start=None):
    start = Path(start or Path.cwd()).resolve()
    for folder in (start, *start.parents):
        if (folder / "Data").is_dir() and (folder / "Src").is_dir() and (folder / "Notebooks").is_dir():
            return folder
    raise FileNotFoundError("Could not locate the repository root.")

PROJECT_ROOT = find_project_root()
TABLES_DIR = PROJECT_ROOT / "Reports" / "week5" / "week5_tables"
FIGURES_DIR = PROJECT_ROOT / "Reports" / "week5" / "week5_figures"
PROCESSED_DIR = PROJECT_ROOT / "Data" / "processed" / "week5"
module_path = PROJECT_ROOT / "Src" / "week5_src_features_dimensionality.py"
spec = importlib.util.spec_from_file_location("week5_features", module_path)
week5 = importlib.util.module_from_spec(spec)
sys.modules["week5_features"] = week5
spec.loader.exec_module(week5)
print("Repository:", PROJECT_ROOT)
print("Source:", module_path)
```

2. Rebuild Week 2/3 data

- Use the original BTS T-100 ZIP archives and Week 2 ingestion code.
- Week 3 target eligibility and chronological 70/15/15 split are reused.
- The model target is the **following month's passengers for the same carrier and directional route**.

In [2]:
```
monthly = week5.load_monthly_without_rewriting_reports(PROJECT_ROOT)
print(f"Carrier-route-month observations: {len(monthly):,}")
print("Reference months:", monthly['Period'].min(), "through", monthly['Period'].max())
assert all(x in monthly.columns for x in week5.CANDIDATE_FEATURES[:5])
```

```
Carrier-route-month observations: 1,882,779
Reference months: 2015-01-01 00:00:00 through 2026-05-01 00:00:00
```

3. Engineer four cross-field features

New feature	Formula	Intended signal
`SeatsPerFlight`	`Seats / DepPerformed`	Mean offered seat capacity per performed departure

New feature	Formula	Intended signal
`PassengersPerFlight`	`Passengers / DepPerformed`	Mean passengers per performed departure
`UnusedSeats`	`Seats - Passengers` if nonnegative	Current-month unused reported seats
`PassengerMiles`	`Passengers * Distance`	Passenger-mile exposure combining demand and route length

Each feature combines **two original input fields**. Zero performed departures produces a missing ratio; negative `UnusedSeats` is treated as invalid/missing rather than silently retained. The next-month target is **not** used in any feature calculation.

In [3]:
```
features = week5.create_cross_field_features(monthly)
display(week5._feature_definitions())
display(features[["Period", *week5.NEW_FEATURES]].head(8))
```

	Feature	Formula	Interpretation
0	SeatsPerFlight	Seats / DepPerformed	Average seats available per performed departure
1	PassengersPerFlight	Passengers / DepPerformed	Average passengers carried per performed depar...
2	UnusedSeats	Seats - Passengers, if nonnegative	Unused reported capacity for the current month
3	PassengerMiles	Passengers * Distance	Passenger-mile exposure combining current dema...

	Period	SeatsPerFlight	PassengersPerFlight	UnusedSeats	PassengerMiles
0	2015-01-01	136.0	28.000000	108.0	4732.0
1	2015-01-01	136.0	28.000000	108.0	4732.0
2	2015-01-01	9.0	6.946746	694.0	72788.0
3	2015-01-01	9.0	6.500000	880.0	70928.0
4	2015-01-01	9.0	3.037037	322.0	10004.0
5	2015-01-01	9.0	3.433962	295.0	11102.0
6	2015-01-01	9.0	3.151899	462.0	2241.0
7	2015-01-01	9.0	2.564103	502.0	1800.0

4. Preserve the Week 3 train/validation/test partitions

In [4]:
```
splits, split_summary = week5.get_week3_splits(features, PROJECT_ROOT)
display(split_summary)
assert splits["Training"]["TargetPeriod"].max() < splits["Validation"]["TargetPeriod"].min()
assert splits["Validation"]["TargetPeriod"].max() < splits["Test"]["TargetPeriod"].min()
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	Mor
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	

5. PCA on nine candidate numeric predictors

The five original numeric features and four engineered features make **nine candidate inputs**. Categorical IDs, dates, and `TargetPassengersNextMonth` are excluded. We apply deterministic `log1p`, impute with **training** medians, remove training-constant columns, standardize with **training** parameters, then fit PCA **on training**. We retain `PC1` and `PC2` as a simple reduced representation. We also report how many components would be required to explain 95% of training variance, but do not assume two components are automatically optimal for forecasting.

In [5]:
```
result = week5.fit_feature_reduction(splits, split_summary)
display(result.dimension_summary)
display(result.component_summary)
display(result.variance_summary)
display(result.loadings)
display(result.split_quality)
assert (result.split_quality["InvalidPCAValues"] == 0).all()
assert (result.split_quality["RowsBefore"] == result.split_quality["RowsAfter"]).all()
```

	Step	Columns
0	Original + cross-field numeric predictors	9
1	Nonconstant training predictors	9
2	PCA reduced representation	2

	Metric	Value
0	Original numeric candidate predictors	9.000
1	Nonconstant training predictors	9.000
2	PCA components retained	2.000
3	Percent training variance captured by PC1+PC2	91.562
4	PCA components needed for 95% training variance	4.000
5	Synthetic observations generated	0.000

	Component	ExplainedVarianceRatio	CumulativeVarianceRatio
0	1	0.737487	0.737487
1	2	0.178133	0.915620
2	3	0.033358	0.948978
3	4	0.026996	0.975974
4	5	0.017887	0.993861
5	6	0.004238	0.998099
6	7	0.001340	0.999439
7	8	0.000406	0.999844
8	9	0.000156	1.000000

	Feature	PC1	PC2
0	Passengers	0.380454	0.052278
1	Seats	0.379492	0.135798
2	DepScheduled	0.324626	0.319002
3	DepPerformed	0.289730	0.504867
4	Distance	0.266082	-0.460612
5	SeatsPerFlight	0.315603	-0.397434
6	PassengersPerFlight	0.325186	-0.380696
7	UnusedSeats	0.326932	0.301029
8	PassengerMiles	0.372667	-0.128487

	Split	RowsBefore	RowsAfter	PCAColumns	InvalidPCAValues
0	Training	1062603	1062603	2	0
1	Validation	239395	239395	2	0
2	Test	264291	264291	2	0

6. Synthetic augmentation decision

No synthetic observations were added.

- Existing BTS dataset is already large, and artificial passenger demand could distort known seasonal dynamics, route discontinuities, and the 2020 demand-regime shift.
- Synthetic tabular augmentation would require separate realism and utility testing and be generated only inside training.
- Task explicitly allows creating cross-field features **and/or** using augmentation, while requiring dimensionality reduction.

In [6]:
```
print("Synthetic data added: 0 rows")
print("PCA fitted on training rows only:", len(splits["Training"]))
```

```
print(f"Two PCA components retain {result.retained_variance_percent:.2f}% of standardized training variance.")
print("Number of PCs required for 95% training variance:", result.components_for_95pct)
```

```
Synthetic data added: 0 rows
PCA fitted on training rows only: 1062603
Two PCA components retain 91.56% of standardized training variance.
Number of PCs required for 95% training variance: 4
```

7. Save compact audit tables, figures, samples

- Full 1.5-million-row outputs are **not** pushed to GitHub by default.
- Small samples and precise fit metadata provide transparent evidence and allow results to be regenerated.
- PCA outputs alone may lose important predictive signals.

In [7]:
```
week5.save_outputs(result, PROJECT_ROOT)
print("Tables:", TABLES_DIR)
print("Figures:", FIGURES_DIR)
print("Samples:", PROCESSED_DIR)
for path in sorted(TABLES_DIR.glob("*")):
    print("  ", path.name)
display(result.reduced_samples["Training"].head())
```

```
Tables: C:\Reports\week5\week5_tables
Figures: C:\Reports\week5\week5_figures
Samples: C:\Data\processed\week5
```

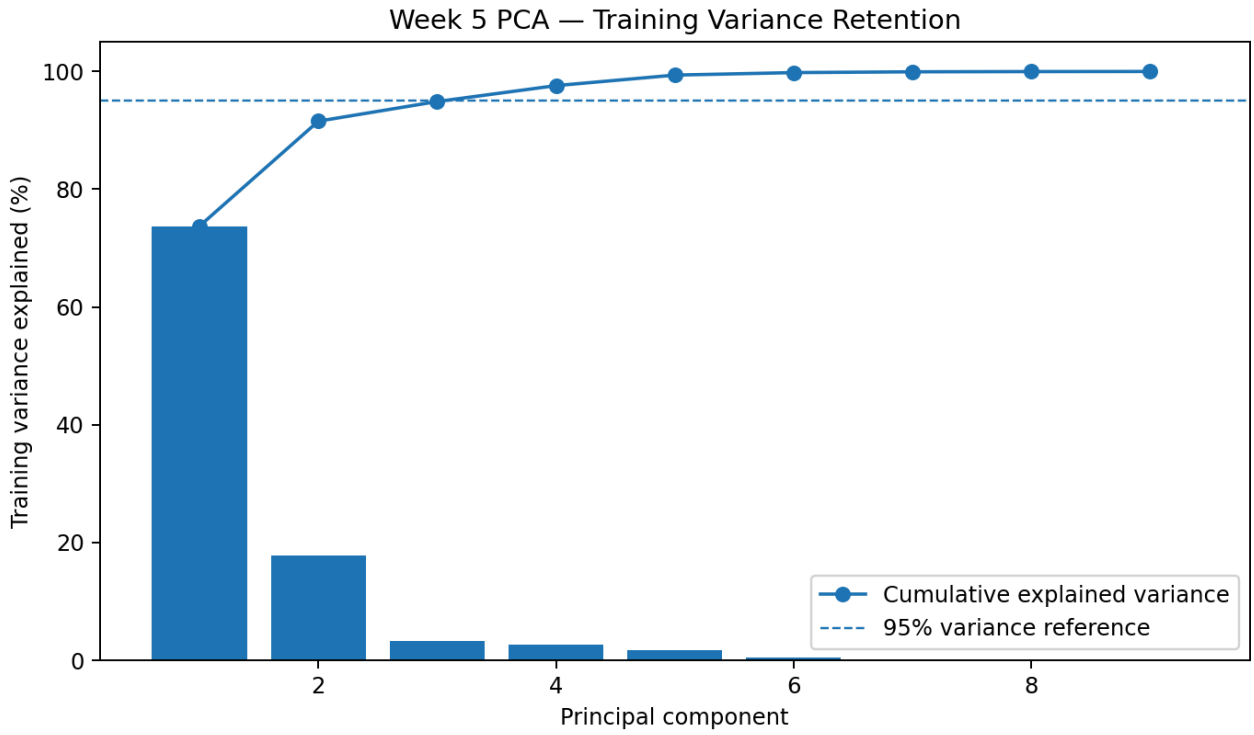
```
  week5_component_summary.csv
  week5_dimension_summary.csv
  week5_feature_definitions.csv
  week5_missingness_audit.csv
  week5_pca_explained_variance.csv
  week5_pca_loadings.csv
  week5_split_quality_checks.csv
  week5_split_summary.csv
  week5_transform_metadata.json
```

	UniqueCarrier	Origin	Dest	Period	TargetPeriod	TargetPassengersNextMonth	PC1	PC2
0	0JQ	GCN	LAS	2015-01-01	2015-02-01	409.0	-3.644553	-1.839158
1	0JQ	LAS	GCN	2015-01-01	2015-02-01	386.0	-3.644553	-1.839158
2	1DQ	ACK	HYA	2015-01-01	2015-02-01	2530.0	-0.687680	4.277715
3	1DQ	HYA	ACK	2015-01-01	2015-02-01	2392.0	-0.646942	4.357466
4	1PQ	HNL	LUP	2015-01-01	2015-02-01	161.0	-2.824769	2.735541

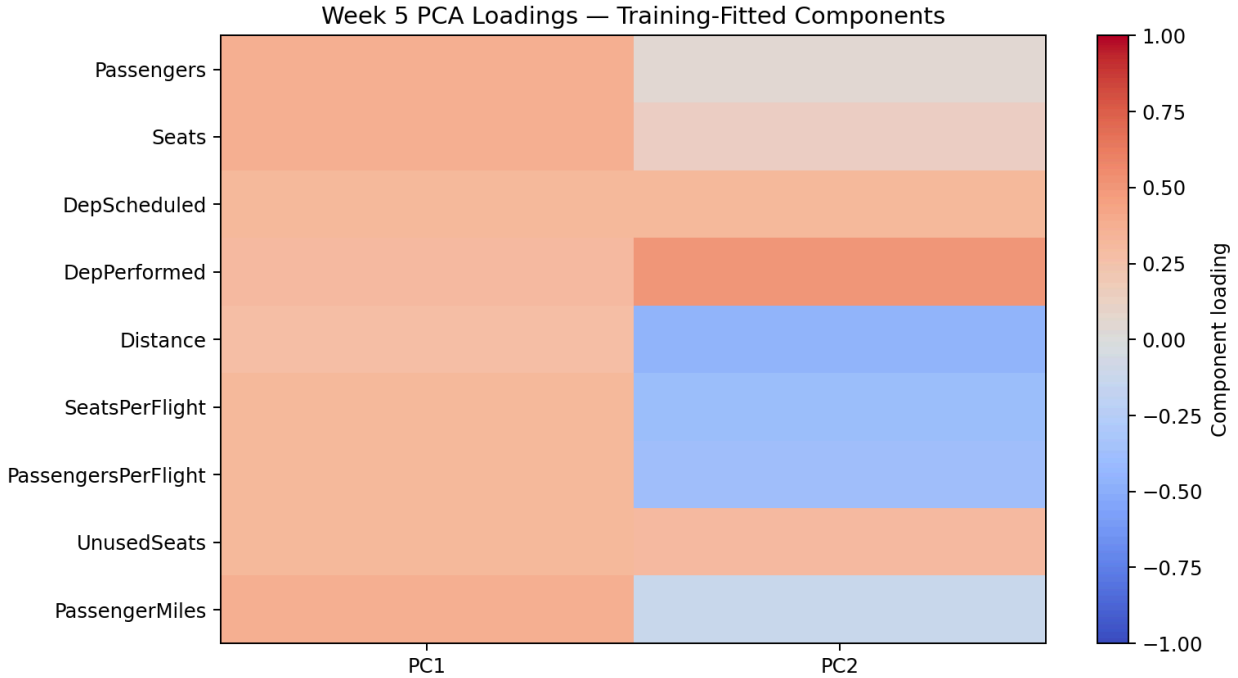
8. Inspect training-only PCA charts

In [8]:
```
for name in [
    "week5_figure_pca_explained_variance.png",
    "week5_figure_pca_loadings.png",
    "week5_figure_dimension_comparison.png",
]:
    path = FIGURES_DIR / name
    if path.exists():
        print(name)
        display(Image(filename=str(path)))
```

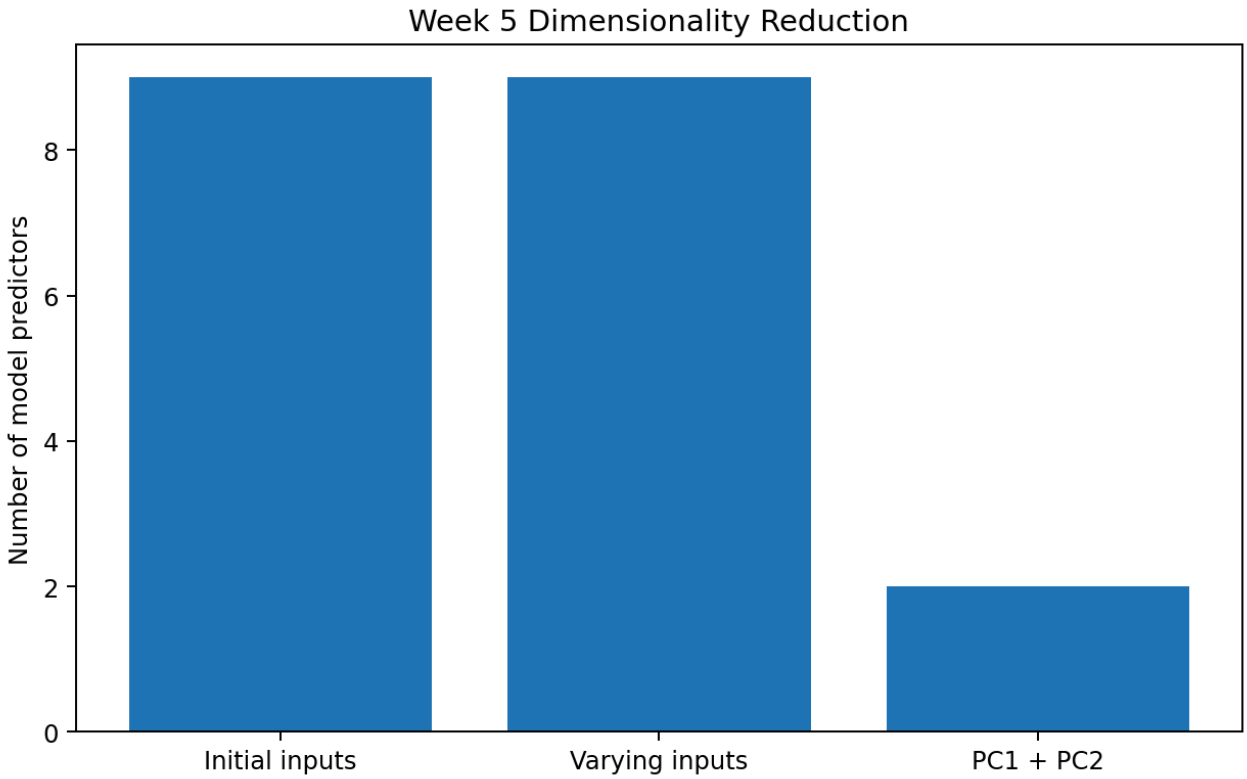
```
week5_figure_pca_explained_variance.png
```



week5_figure_pca_loadings.png



week5_figure_dimension_comparison.png



9. Final validation and completion

- This is feature engineering and dimensionality reduction
- This is **not a fitted passenger-demand forecasting model**.
- The test labels were never used to select inputs or components.
- The fraction of *training variance* explained by PCA is not a forecast accuracy metric.

In [9]:
```
assert (TABLES_DIR / "week5_transform_metadata.json").is_file()
assert (FIGURES_DIR / "week5_figure_pca_explained_variance.png").is_file()
for name in ("training", "validation", "test"):
    assert (PROCESSED_DIR / f"week5_{name}_reduced_sample.csv").is_file()
assert len(result.kept_features) > 2
print("All Week 5 checks passed.")
print("===== WEEK 5 END =====")
```

```
All Week 5 checks passed.
===== WEEK 5 END =====
```